

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 7, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 7, 2026	41 pages

Tiivistelmä — Referat — Abstract

Context. Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service modelled on a real Nokia 5G base-station control-plane component (gNB-CU-CP). Average-throughput comparisons leave four dimensions open: tail latency, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with identical algorithms, concurrency model, allocator, and build settings. A synthetic workload is increased to overload, both versions pass the same parity tests, and the evaluation adds fault injection and a foreign function interface (FFI) prototype.

Results. (1) Safe Rust rejects all seven injected memory-safety and data-race defect classes at compile time. In C++ the same seven build cleanly and show up only at runtime, if a sanitiser catches them. (2) Seven further classes fail in both languages. On two of them C++ tooling reports what Rust does not. (3) C++ can call the Rust service through a thin C interface. All `unsafe` stays there, and any extra time on a real call is at most 10.91%. (4) A full request spends only 0.250% on language work. Passing work between threads takes the rest, so that clock cannot attribute a gap to C++ or Rust. Timed on its own, Rust's operation is 1.6× faster. (5) Below overload the full path is a few microseconds faster in Rust. The Rust versions keep up with the offered load to 448 kops/s, the C++ versions only to 392. Once neither can keep up, C++ still finishes 1.7× as much work as Rust in 20/20 mixed-order repeats with both allocators.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. A nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures iii

List of acronyms v

1 Introduction 1

1.1 Motivation and industrial context 2

1.2 Problem statement 6

1.3 Research questions and hypotheses 7

1.4 Contributions, results, and limitations 8

2 Background: Systems Languages for Latency-Critical Software 11

3 Problem Context: UE Context Management and Predictability 15

4 Research Methodology 17

5 Design and Implementation 19

6 Experimental Setup and Evaluation 21

7 Discussion 27

8 Conclusion and Future Work 31

Statement on the Use of AI Tools 33

Bibliography 35

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 11
2.2 Why the tail, not the mean, governs RAN deadlines 12
2.3 The same use-after-free in C++ and in safe Rust 12
2.4 What the neighbouring studies control, and what they leave varying . . 13

3.1 UE-context lifecycle state machine 15
3.2 The overload cliff 16

5.1 Shared-nothing, thread-per-shard architecture 19

6.1 The two workloads, and the lifecycle the mixed one walks 22
6.2 The load sweep: tail latency and achieved throughput 23
6.3 The end-to-end operation decomposed, against what the sweep can resolve 23
6.4 Divergence past saturation 24
6.5 Where the guarantee stops: the outcome matrix 25
6.6 Resource use, and the two effects inside peak RSS 26
6.7 The C++-to-Rust FFI migration seam 26

7.1 Where Rust's compile-time guarantee stops 28

List of acronyms

Each acronym below is also written out where the text first uses it.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
AWS	Amazon Web Services
CISA	Cybersecurity and Infrastructure Security Agency
CPU	Central processing unit
EU	European Union
FBI	Federal Bureau of Investigation
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane: the system class studied here
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now re-implemented as programs on general-purpose machines. The radio access network (RAN), long the most hardware-bound part of the stack, is part of that shift.

In 5G, the base station, formally the next-generation NodeB (gNB), is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device, each of which the standards call a user equipment (UE), runs Radio Resource Control (RRC) signalling [4], and coordinates mobility and handovers, all within strict latency limits.

Load at a busy site is high. A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete inside the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage or a tail-latency spike here is not a local defect. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [10, 11], with performance intended

to be competitive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane software this work is about. *UE Context Management (UECM)* is the specific subsystem reimplemented and measured. *Tail latency*, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [6, 7]. Every acronym is written out where the text first uses it, and the List of acronyms in the front matter collects them.

1.1 Motivation and industrial context

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN itself. Moving control-plane functions onto general-purpose servers raises, rather than lowers, the premium on software quality: the software is longer-lived, evolves continuously, runs at higher concurrency, and must meet its deadlines without the determinism that purpose-built hardware once provided. This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces [3], so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the ease with which a large codebase can be evolved safely) and no longer on purpose-built hardware. In this setting a latency regression or a memory-corruption defect is not a localised bug but a network-wide reliability and security risk, and a language-selection question stops being academic and becomes a concrete engineering decision.

The second trend is that artificial intelligence (AI) is moving from a tool used to tune the network into a workload the network is built to carry, and the next generation is being specified that way from the start. International Mobile Telecommunications-2030 (IMT-2030), the International Telecommunication Union (ITU) framework for 6G, lists artificial intelligence and communication among its six usage scenarios and

ubiquitous intelligence among its four design principles [13]. The architecture forming around that is AI-RAN, in which telecommunications and AI workloads are orchestrated together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms [14]. The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [15]. That changes what a language choice has to buy for a control plane. Its software no longer has the host to itself. It contends for central processing unit (CPU), memory, and cache with inference work whose arrival pattern it does not control, and a co-tenant that takes capacity away lowers the load at which the control plane saturates. Saturation is where the two languages separate most sharply in the measurements of Chapter 6. Memory footprint and behaviour past that point are becoming selection criteria in their own right, and neither is settled by the architecture above the code.

The third trend is an industry-wide shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [8, 9, 16, 17]. The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all [18]. This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ [19, 20]. In the market this subsystem is sold into it has also become a regulatory matter. The European Union (EU) Cyber Resilience Act places secure-by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 [21]. It names no programming language, and it would be wrong to read it as requiring one. What it changes is that the recurring defect classes of a memory-unsafe language become a compliance exposure as well as an engineering one. Rust in particular has moved from a niche language into operating-system kernels, cloud infrastructure, and increasingly into telecommunications. Production hypervisors such as Amazon Web Services (AWS) Firecracker, which underpins its serverless platform, are already written in Rust [22]. Yet the evidence base for Rust's suitability rests largely on general-purpose software and on average-case performance benchmarks. The specific question a telecom vendor must answer, whether Rust can meet the tail-latency and

predictability requirements of a RAN control plane, whether it genuinely prevents the defect classes that plague the existing C++ code, and what it costs to adopt it incrementally inside a large C++ codebase, has not been answered with direct, controlled evidence. Providing such evidence for a representative subsystem is the motivation for this work.

Industrial setting. This thesis is carried out at Nokia, one of the small number of vendors that build, maintain, and evolve production RAN software end to end. That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [23, 24]. The subsystem it studies, UE Context Management (UECM), is modelled on a real gNB-CU-CP control-plane component rather than a textbook example.

The gNB-CU-CP is not a minor component. In the disaggregated architecture of the 3rd Generation Partnership Project (3GPP) it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle [1, 4], under the millisecond-scale procedure budgets the 5G system places on control signalling [5] and at the event rates and connected-device counts a deployed network sustains. Chapter 2 places it in the wider architecture. Because it is written in C++, it inherits both that language's performance and its memory-safety and concurrency hazards. That is precisely where the three trends above converge, and software of this class is shipped, evolved, and operated across live operator networks, so the language-selection decision is a production concern rather than a hypothetical one.

UECM is a deliberate choice of study object within that setting, for the reasons Chapter 3 sets out: it is small enough to reimplement faithfully and reason about in full, yet it carries the properties that make control-plane code hard. The implementation is driven entirely by a public, frozen specification rather than by proprietary code, so the comparison stays reproducible by readers outside Nokia.

A decision with a generational horizon. A language choice in software of this class is not revisited often, and it outlives the generation it was made for. The next one is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against an ITU calendar that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 [25]. That work explicitly

covers migration from and interworking with 5G, so a control plane written for the current generation is maintained alongside the next one rather than retired when it arrives. Nor do the next generation's objectives relax what motivates this comparison. IMT-2030 names security and resilience as one of its four design principles, expects the system to be secure by design, defines resilience as the ability to keep operating correctly during and after a disruptive event, and carries the ultra-reliable low-latency scenario forward in strengthened form [13]. Freedom from whole classes of defect and predictable behaviour under disturbance are being written into the objectives of the next generation rather than left to implementers.

The policy dates named above fall inside the same window. The Cybersecurity and Infrastructure Security Agency (CISA) and the Federal Bureau of Investigation (FBI) classify the development of new product lines in a memory-unsafe language as a bad practice where the product serves critical infrastructure, a scope mobile-network infrastructure sits squarely inside, and they exempt an existing product from the memory-safe roadmap expectation only when its announced support ends before January 2030 [26]. A vendor that defers the language question until the next generation is specified will not thereby have avoided making it. It will have made it by default, in the incumbent language, for another decade of deployments.

What this thesis contributes to that decision is bounded, and the boundary is worth stating plainly. The measurements here are taken on a subsystem modelled on a current-generation component and evaluated against requirements that are already published. No claim is made about requirements that have not yet been written. What crosses the generational boundary is not a latency figure but the language-level comparison itself: Which defect classes a compiler can rule out, what the safety model costs, and what incremental adoption inside a large C++ codebase involves. Those questions have the same answers whichever generation the specification above them names, and they are cheapest to answer before the next codebase is started rather than after.

A concrete failure mode. To make the stakes concrete, consider a mobility storm in which many UEs hand over at once. A single mishandled pointer, a UE context freed on release while an in-flight signalling message still references it, produces a use-after-free. In C++ this compiles cleanly and may pass ordinary testing, only to corrupt an unrelated UE's state or crash the worker thread under precisely the high-concurrency

conditions the system exists to handle. The same latent race can equally show up as an unpredictable tail-latency spike instead of an outright crash. In safe Rust the same design does not compile: ownership and borrowing reject the dangling reference at build time. This one scenario captures why the thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload) rather than average throughput alone.

1.2 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 27], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.3 Research questions and hypotheses

The thesis is organised around four research questions, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

The questions are approached with three prior expectations, stated here so that the evaluation can confirm or refute them instead of merely describing results after the fact. **H1:** For this workload the two languages are performance-equivalent in the operationally relevant region below saturation, with any difference confined to the far tail. **H2:** Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ leaves to best-effort, after-the-fact detection. **H3:** The run-time cost of C++-to-Rust interoperability is negligible relative to the cost of the operations invoked, so that the true cost of incremental adoption is engineering effort rather than performance. H2 is stated so that it can fail. The defect catalogue behind RQ2 spans classes safe Rust is expected to prevent and classes it is expected not to, so the study can locate the boundary of the guarantee instead of only confirming it. These hypotheses are revisited against the evidence in Chapter 7.

1.4 Contributions, results, and limitations

The thesis makes six contributions. Two are artifacts: a reproducible benchmarking framework for telecom control-plane workloads, and a frozen shared specification with two parity-tested implementations of a RAN control-plane subsystem, one in C++ and one in Rust. Three are results. The first is a comparison of latency and resource behaviour that divides at the saturation knee, where the ordering between the two languages reverses. The second is a defect-class taxonomy from controlled fault injection that marks where the compile-time guarantee stops, in both directions. The third is a bound on what a comparison of this shape can attribute to a language at all, obtained by decomposing an end-to-end operation into the work the language performs and the cross-thread handoff around it. The last is a set of engineering and migration guidelines for language selection in cloud-native RAN evolution (Chapter 7). Neither the reversal nor the attribution bound was anticipated. Both were found while reading results the study had planned to report differently, and Chapter 7 records what the first of them cost H1.

The findings divide three ways. On *safety* (RQ2), every injected memory-safety and data-race defect class is prevented at compile time in safe Rust. The same defects all compile in C++, and each is caught, if at all, only at run time and only under the right sanitiser, or on one class under none of them. Seven boundary classes were chosen deliberately, and neither language prevents them. On two of those the C++ tooling is ahead, and on one the defect that safe Rust rules out returns as soon as the program writes `unsafe`. On *migration* (RQ4), the FFI seam is small and keeps the core free of `unsafe`. Measured against the same operation called natively it costs at most 10.91% of one operation, so the real adoption cost is application programming interface (API) design and not run-time overhead. Three costs beyond the seam are measured for the same reason, and two of them come back against the received wisdom: most of the Rust build's extra cost is compiling third-party code and not the compiler's own work, and the binary-size ordering depends on which linkage policy the two toolchains are held to rather than on the languages. Both of these results are final.

On *performance* (RQ1, RQ3) the answer divides twice, and drawing both divisions correctly is where most of the analysis went. The first division separates what the sweep measures from what it can attribute. An end-to-end operation in this design is 0.250% language work and a cross-thread handoff takes the rest, so a matched-pair

comparison on that path bounds the design and not the language. The limit holds whichever way such a comparison comes out, and it reaches any study that reports an end-to-end median without measuring the share. Timed apart from the handoff, the operation is 1.6× faster in Rust, which the sweep cannot see.

The second division is by regime. Below saturation the two implementations are equivalent end to end in central behaviour, by a margin no deployment would notice. The paired test does resolve a median difference of 1.3 μs in Rust's favour under both allocators, on an end-to-end round-trip of 70 μs, so the equivalence is practical and not statistical. The far tail in the same regime resolves on the reported host and favours Rust. Pooled over 3 360 051 to 3 360 084 operations per variant, the two languages' p99.99 values do not meet, and the widest gap between variants is 87 μs. A second bare-metal campaign at another shard count agrees on the direction (Chapter 6). Past saturation the ordering reverses, and where each implementation stops meeting its offered rate is itself a difference: the Rust arms track to 448 kops/s and the C++ arms to 392. Deeper still, where all four have collapsed, the C++ build sustains 1.7× the throughput of the Rust build in 20/20 interleaved repeats under both allocators, and the Rust build spends 1.8× the CPU on each operation it does complete, which is what rules out the reading that it is simply doing less. On resources the Rust process holds a few MiB more resident memory, again under both allocators, in the same direction and the same order of magnitude on all three machines measured. How it compares against the cost of the allocator choice does not travel: the two are comparable on bare metal and the allocator's was an order of magnitude larger on the virtualised host this study used first, so the comparison between them is a property of the machine rather than of the allocators.

Four limitations bound all of this: a synthetic workload, one simplified subsystem, single-author implementation bias, and an allocator footprint whose size moved with the host. Chapter 7 treats each in detail. The intended readers are the vendor making the language decision, the standards discussion around memory-safe roadmaps [19, 20], and anyone repeating a comparison of this shape, who can reuse the specification, the pair, the harness and the catalogue.

The scope is drawn deliberately to protect the internal validity of the language comparison. In scope are the UE Context Management functionality of Chapter 3, the shared-nothing concurrency design, the load sweep to overload, the fault-injection catalogue, and the FFI prototype. Explicitly out of scope are the full gNB system and its

remaining control-plane domains, physical layer (PHY) and layer-2 processing and the data-heavy user plane, real captured traffic traces (as opposed to a spec-justified synthetic workload), managed-language alternatives (excluded on the tail-latency grounds argued in Chapter 2), and production deployment and operational concerns such as upgrade, observability, and multi-tenancy. These exclusions trade external validity for the controlled conditions a fair language comparison needs, and each is revisited as a limitation in Chapter 7.

2 Background: Systems Languages for Latency-Critical Software

In the next-generation radio access network (NG-RAN) architecture the base station (gNB) is split into a central unit and a distributed unit. The central unit is split again into a control plane and a user plane. UE Context Management sits in the control plane, the gNB-CU-CP, and talks to its neighbours over the standardised F1, E1, and NG interfaces [1]. Figure 2.1 places it there.

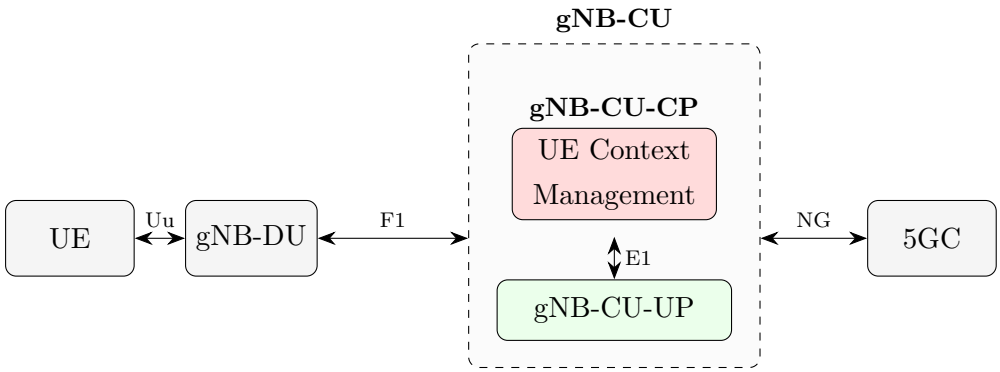


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. The gNB is split into a distributed unit (gNB-DU) and a central unit (gNB-CU), and the central unit is split again into control-plane (gNB-CU-CP) and user-plane (gNB-CU-UP) functions, connected by the standardised F1, E1, and NG interfaces. UE Context Management (highlighted) is the per-UE control-plane state manager inside the gNB-CU-CP that this thesis reimplements and measures.

Deadlines are met or missed because of predictability, not average speed. The metrics that matter are the p99 and p99.9 latencies and the jitter around them [6]. They must be measured past the saturation point. They must also be recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests [28, 29]. Figure 2.2 shows why the distance between median and tail matters once one procedure runs several operations.

C++ offers abstractions that cost nothing at run time, deterministic cleanup through the Resource Acquisition Is Initialization (RAII) idiom, and no garbage collector. Safety remains the programmer's responsibility [30]. Sanitisers report only the errors that a given test run happens to reach.

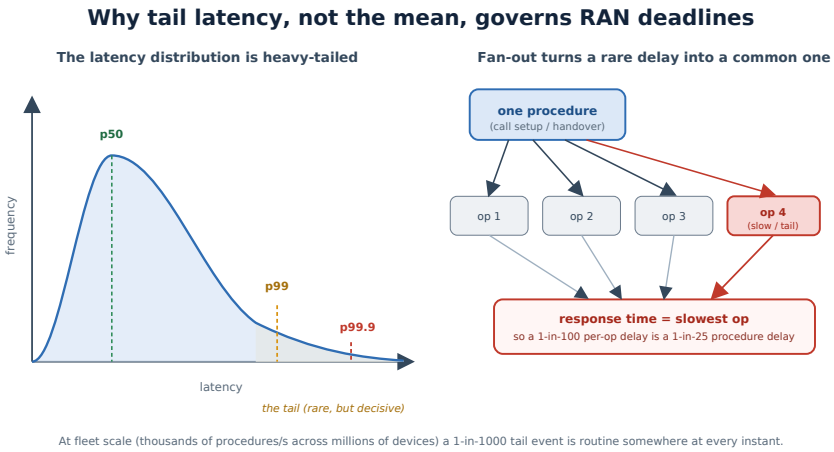


Figure 2.2: Why the tail, not the mean, governs RAN deadlines. On the left, most operations finish quickly and a few take far longer, so the p99 and p99.9 percentiles sit well to the right of the median. On the right, a procedure such as a call setup runs several operations and finishes only when the slowest one does, so a delay hitting one operation in a hundred reaches a much larger share of procedures.

Rust has the same run-time model and also no garbage collector. It checks memory safety and freedom from data races at compile time, using ownership, borrowing, and thread-safety traits such as `Send` [10, 31]. Code that needs to step outside those rules must be marked `unsafe`. That keeps such code small and easy to find. Figure 2.3 puts the introduction's use-after-free beside its C++ form.

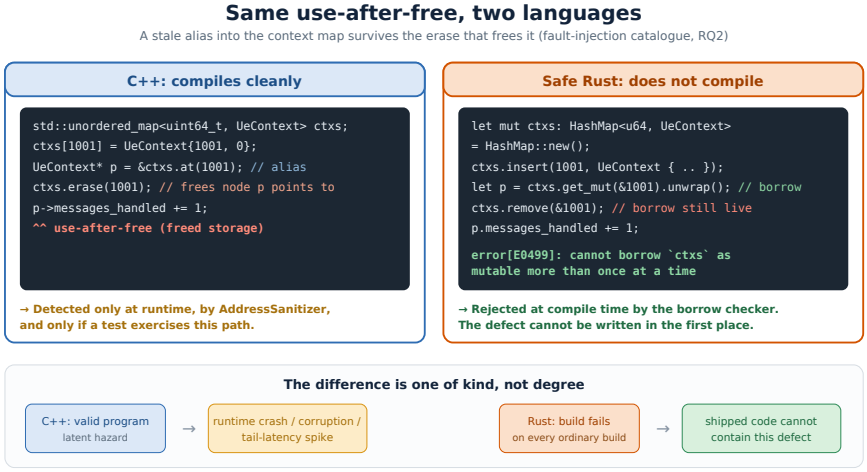


Figure 2.3: The same use-after-free defect in C++ and in safe Rust. The stale alias into the context map survives the `erase`/`remove` that frees it, which in C++ compiles and is caught only at runtime, and only under a sanitiser on a path a test exercises. Safe Rust's borrow checker rejects it at compile time (`E0499`), so the defect cannot be written at all.

The comparison is C++ against Rust. Garbage-collected languages are left out because their pauses are incompatible with the latency targets. The concurrency model is held fixed in both implementations. The C application binary interface (ABI) is treated as the realistic adoption path. No earlier study isolates the language this way for a stateful, concurrent subsystem [32, 33]. The five closest studies by domain were all carried out at Ericsson, and none is peer-reviewed or indexed [32, 33, 34, 35, 36]. Repetition inside one organisation is not generality. Figure 2.4 sets those five against the properties a controlled answer would need.

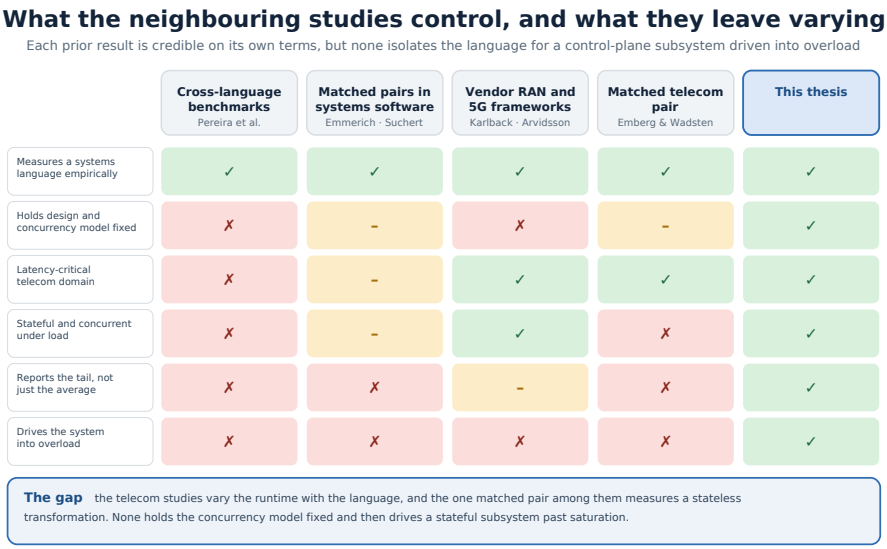


Figure 2.4: What the neighbouring studies control, and what they leave varying. Each row is a property that a controlled answer to the research questions would need and each column is a study, and no column holds all of them. Holding the design and the concurrency model fixed for a stateful, concurrent subsystem and then driving it past saturation is what this thesis adds, so the contribution is the control and not a new measurement technique.

3 Problem Context: UE Context Management and Predictability

UE Context Management owns the lifecycle of one context per connected device. Each context moves through four states, Idle, Connected, Reconfiguring, and Released, driven by a small set of lifecycle operations. An operation that is illegal in the current state returns a typed error. It is not carried out. Figure 3.1 gives the state machine both implementations encode.

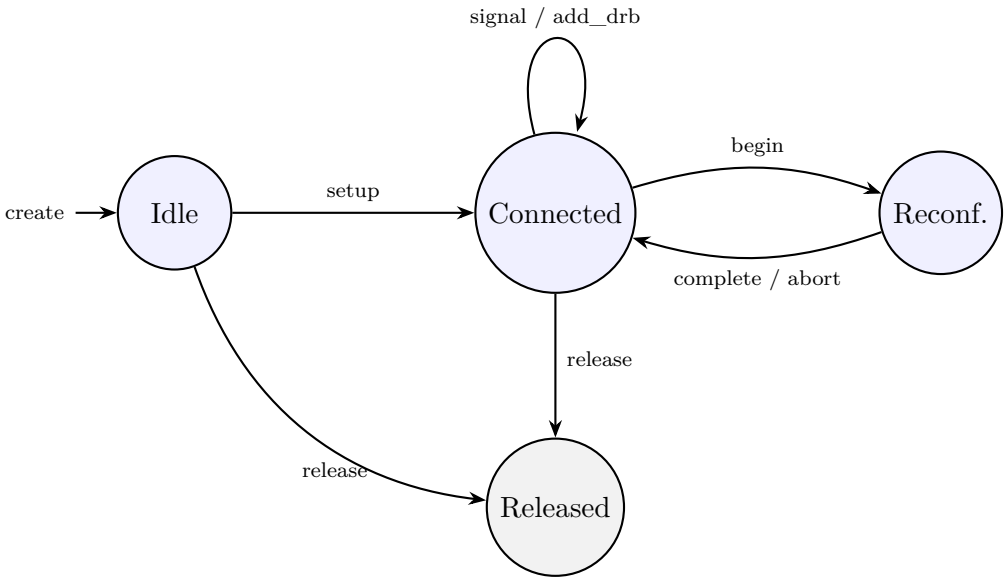


Figure 3.1: UE-context lifecycle state machine, encoded identically by both implementations. `create` allocates a context in `Idle` and `setup` connects it, `signal` and `add_drb` are handled in place while `Connected`, a reconfiguration moves through `Reconfiguring` and back on `complete` or `abort`, and `release` tears the context down. Operations invalid for the current state return an `OpError` instead of changing state.

The subsystem was chosen because it is representative and still small enough to keep under control. It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets [5, 37]. It is still simple enough to specify precisely and to test for identical behaviour in two languages.

Seven defect classes fall inside Rust's guarantee. Each has a natural form in this subsystem: use-after-free, double-free, iterator invalidation, data race, unguarded shared

mutation, use-after-move, and a thread-unsafe handle shared between workers. The guarantee rests on aliasing control and data-race freedom [31]. The catalogue is this study's own.

Seven further classes are included on purpose from outside that guarantee: a lock-order deadlock, a lost update, a leak through a reference cycle, an integer overflow in a release build, the blast radius of a failed shard worker, a double release across the C-ABI seam, and a lock guard that is never dropped. They keep the study from confirming a foregone conclusion. They show where the guarantee stops. Two of them return the opposite of the expected result. The C++ tooling reports what Rust's does not.

Scale is what makes the tail matter. One procedure triggers many operations. One site serves many devices. A p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [6]. That is why the load sweep is pushed past the point where the service starts to fall behind. Figure 3.2 sketches what lies beyond it.

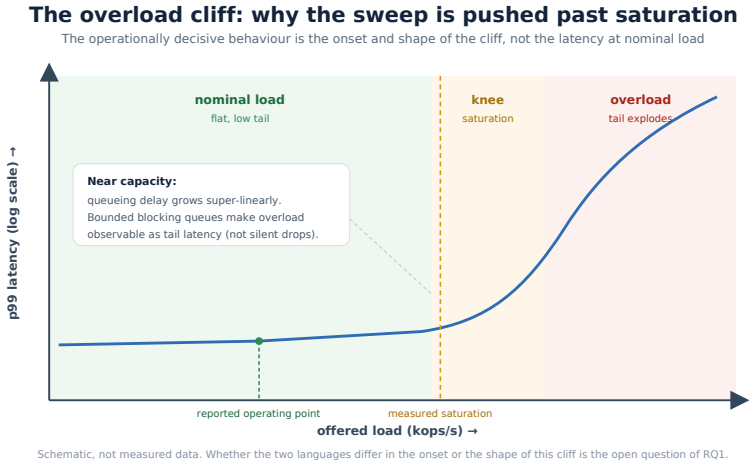


Figure 3.2: The overload cliff. Tail latency stays low and flat while offered load is below service capacity, then climbs steeply once the system is driven past saturation and requests queue behind the bounded, blocking queues. The figure is a schematic of the phenomenon and not a plot of this study's data, and whether the two languages differ in where the cliff begins or in what they sustain beyond it is what RQ1 asks and Chapter 6 answers.

4 Research Methodology

The approach is empirical and constructive. Instead of comparing two systems that already exist and differ in many ways, the study builds a matched pair of implementations from one frozen specification. The language is then the only systematic difference between them. Both use an identical thread-per-shard concurrency model. That structuring itself changes tail latency [38].

RQ1 and RQ3 are answered with an open-loop load sweep from light load into overload on pinned cores. Open-loop means the generator keeps to its send schedule instead of waiting for replies. Latency is recorded in identically configured high-dynamic-range (HDR) histograms [29]. Resource use is sampled through the operating system. The reported sweep runs on a dedicated AMD EPYC 7443, simultaneous multithreading off, frequency governor fixed at performance, with `nohz_full` on the pinned cores and no hypervisor. That host resolves the overload divergence, the resource comparison, and the far tail below saturation. The same design was first measured on a Intel(R) Xeon(R) CPU E5-2680 v4 guest (kvm) that could not resolve the far tail: there the widest interval on one variant's p99 exceeded the spread between all four by a factor of five, so no ordering could be read however many repeats were run. A third sweep on a second bare-metal AMD EPYC 7443P uses a different shard count, load grid, repeat count and toolchain, so it tests whether the findings survive a change of design rather than a change of machine.

RQ2 is answered by controlled fault injection [39]. A fixed catalogue of defects is written in both languages. The expected outcome of each one is recorded in advance. A surprise then shows up as a discrepancy, not as something explained away afterwards.

RQ4 is answered by a working prototype in which C++ drives the Rust core through a foreign-function interface. Cross-language memory-management defects concentrate at that seam [40]. It yields three measurements: How large the seam is, how much `unsafe` code it needs, and what one call across it costs.

Fairness is the central experimental control. The specification is frozen. Allocator and optimisation settings are matched. Every point where the two languages had to differ is written down in a forced-idiom log. Reporting follows the guidance that a single figure invites conclusions run-to-run variation does not support [41]. Medians come with

nonparametric bootstrap confidence intervals [42]. Exact paired permutation tests are corrected for multiple comparisons by the Holm procedure [43]. All data is synthetic, so no data-protection concerns arise.

5 Design and Implementation

Nothing is implemented until the specification is frozen. It fixes the public API, the UE state machine, the data structures and their algorithmic strategy, the locking and sharding granularity, and the concurrency model. Both implementations conform to it. The comparison then measures the language, not a difference of design or of skill. The specification is itself a deliverable. It is the study's main control against implementation bias.

One concurrency model is used on both sides, shared-nothing and thread-per-shard. That structuring is known to change tail latency by itself [38]. The identifier space is partitioned into shards. Each shard is owned by a single worker thread that alone reads or mutates it, so no lock guards context state. Callers submit a request to the owning shard's bounded queue and wait on a single-use completion handle. The queue blocks when it is full. That is deliberate. A drop policy would hide overload, and this study needs overload to appear as latency. The only shared mutable state either version permits is one atomic counter. No control flow depends on its value. Figure 5.1 shows the resulting data flow, with one multi-producer, single-consumer (MPSC) queue per shard that its worker drains first in, first out (FIFO).

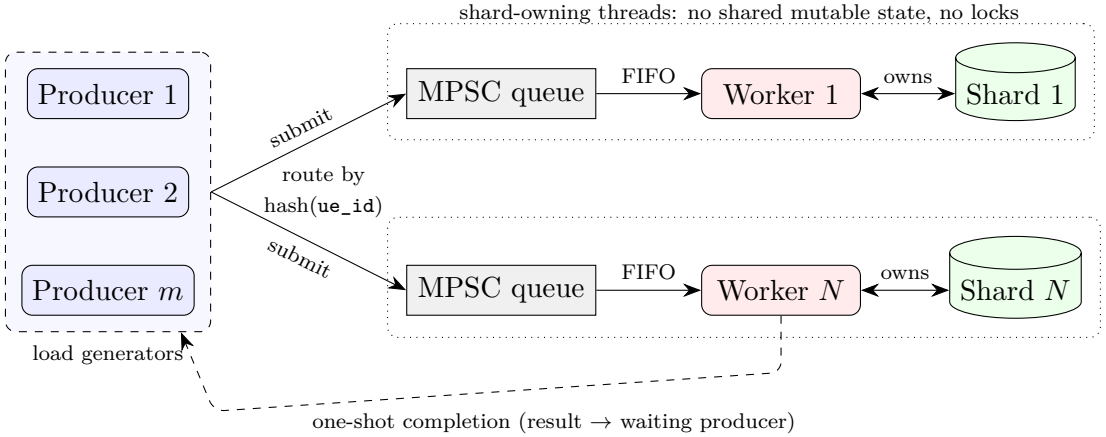


Figure 5.1: Shared-nothing, thread-per-shard architecture. Producers submit request messages to a shard's bounded, blocking queue, selected by a pinned hash of the `ue_id`, and wait on a single-use completion handle. Each shard is read and written by exactly one worker thread, so there are no locks on context state, and blocking on a full queue is what makes overload observable as tail latency.

Fairness is enforced below the design as well as in it. Both builds link the same

`jemalloc` and keep a system-allocator build as a control. Both compile at matched optimisation. Both record latency through one HDR histogram algorithm configured identically [29]. Hashing is pinned on both sides, SplitMix64 for shard assignment and FNV-1a, the Fowler–Noll–Vo hash, inside the map. C++'s default hash of an integer is typically the identity function. Rust's is cryptographically strong. Left as they came, the two standard libraries would have been the measurement.

Where the languages cannot satisfy a requirement the same way, the divergence is written into a forced-idiom log and then measured. Rust takes its bounded blocking queue from `sync_channel`. C++ through C++23 has no such primitive. It assembles one from a mutex and two condition variables. C++ has no payload-carrying enumeration in this idiom either. A request becomes a tagged struct holding every argument field any operation might need, 160 bytes against Rust's 96. The caller answers through a `variant` it has to unwrap. The net result is a Rust core 18% longer in code lines whose total cyclomatic complexity is 15% lower [44]. The branchiest function in either core is the C++ worker loop that dispatches that `variant`. The Rust library contains no `unsafe` at all.

Parity is tested and not assumed. Language-neutral vectors run in both implementations and their transcripts are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third implementation in Python acts as an independent oracle. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of roughly 793 000 generated operations found no divergence. The transition matrix is enumerated exhaustively instead of sampled. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri. Line coverage is reported. A clean sanitiser result is therefore bounded by the code that actually ran.

6 Experimental Setup and Evaluation

The workload is synthetic and spec-justified. A deterministic driver walks each UE through the connection lifecycle the 3GPP specifications define. Its draw order is pinned, so the per-operation-kind counts come out identical in both languages for a given seed. That doubles as a parity signal, and Figure 6.1 shows the two workloads and the procedure sequence the mixed one walks. Each sweep point is repeated 20 times. The four language and allocator variants are interleaved inside every repeat, so thermal or background drift falls on all of them alike. Every point is a median with a bootstrap 95% confidence interval [41, 42]. The representative operating point is fixed at 56 kops/s and not derived. The rule this study began with, the busiest swept load holding p99 under 5 ms, located the virtualised host's noise floor and on a quiet host climbs until it collapses onto the saturation knee, so it is kept as a diagnostic only. The measured throughput plateau is 392 kops/s, the lowest of the four, reached at an offered 392 kops/s. All of it runs on a dedicated AMD EPYC 7443 pinned to cores 2–17. The frequency governor is fixed at performance with boost disabled. The kernel runs tickless on those cores. There is no hypervisor. Isolating the cores with `isolcpus` is omitted on purpose, because it would collapse the pinned set onto one core. The same design was first measured on a Intel(R) Xeon(R) CPU E5-2680 v4 guest that could not resolve the far tail. A second bare-metal host at a different shard count confirms that the directions survive a change of design.

Below saturation the four variants sit within 2 μs of one another at the median, 2.1%. The paired test resolves a difference of 1.3 μs in Rust's favour under both allocators. Figure 6.2 draws the sweep behind that, tail latency and achieved throughput against offered load. Higher up the distribution the ordering can now be read. The widest interval on a single variant's own p99 spans 1 μs against 2 μs of spread between all four, so the uncertainty about one is smaller than the gap between them. On the virtualised host the same two quantities were 1480 μs against 293 μs, the wrong way round. The far tail resolves here too, from the repeats' histograms pooled into 3 360 051 to 3 360 084 operations per variant, and both Rust variants sit below both of the others by up to 87 μs. A second bare-metal host at another shard count agrees, running 147–228 μs

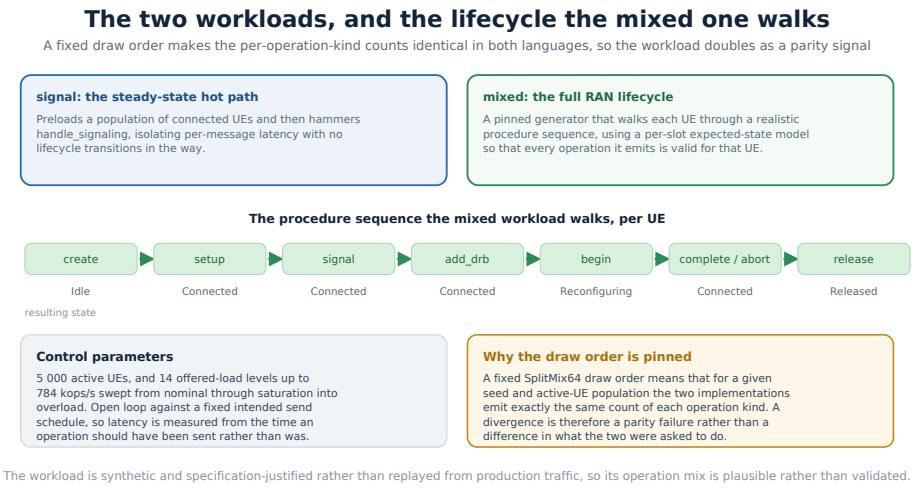


Figure 6.1: The two workloads. The *signal* workload isolates the steady-state hot path, while the *mixed* workload walks each UE through the full RAN procedure sequence, with a per-slot expected-state model ensuring that every operation it emits is legal for the state its context is actually in. Because the SplitMix64 draw order is pinned, a given seed and active-UE population produce identical per-operation-kind counts in both languages, so a divergence is a parity failure rather than a difference in what the two implementations were asked to do.

for the C++ variants against 88–115 μs for the Rust ones, without overlap. A second limit is structural. No host would fix it. An end-to-end operation is 0.250% the work the language performs. The rest is a cross-thread handoff charged to the operating system [45]. This sweep cannot bound a language effect in either direction, which Figure 6.3 sets against what it can resolve. Measured on its own the bare operation is 1.6× faster in Rust. The likely cause is the standard hash map's layout, not anything about safety.

Once the offered rate outruns the service, two differences appear. They stop meeting the offered rate at different loads: the Rust arms hold to 448 kops/s and the C++ arms to 392, so at 448 kops/s Rust still serves 448 kops/s where C++ has collapsed to 162. That rung is described and not tested, because the operating point was found after the data was in hand. Deeper, at 784 kops/s offered, the ordering reverses: C++ sustains 147 kops/s where Rust sustains 84, a factor of 1.7, in 20/20 repeats under both allocators with the Holm-corrected p at the floor of the whole test family. The slower variant is not simply doing less work. It draws 1.8 times the CPU per completed operation, which is the signature of effort spent without completing anything. Isolating the bounded queue and the completion handle on their own does not reproduce that ordering at significance, so the mechanism sits in the wider handoff path. This is the

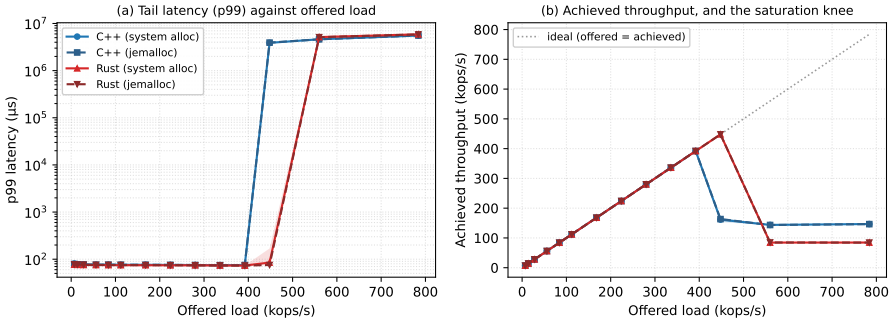


Figure 6.2: The load sweep, both panels against offered load. **(a)** Tail latency (p99) for the four build variants on a log axis, flat across the whole tracking region and then rising by four orders of magnitude in one step of the ladder. **(b)** Achieved throughput against the dotted ideal, which the C++ curves leave after an offered 392 kops/s and the Rust curves after 448, so 448 kops/s is the only load in the sweep at which the two languages differ in *throughput* with Rust ahead. Lines are medians over 20 repeats and shaded bands are bootstrap 95% confidence intervals in both panels.

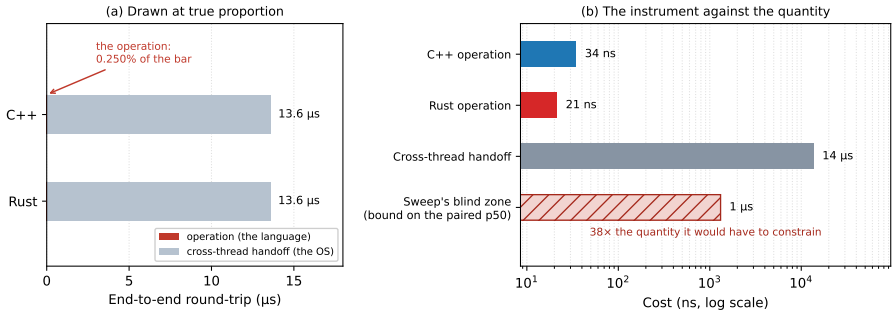


Figure 6.3: Where an end-to-end operation's cost goes, and why the sweep cannot bound a language effect. **(a)** The round-trip at true proportion, in which the operation the language performs is 0.250% of the bar and the remainder is the cross-thread handoff. **(b)** The same quantities on a log axis against the smallest paired difference the sweep can resolve, shown as a blind zone that both operation measurements fall inside by a factor of 38.

largest language effect in the tested family. A comparison that stopped at nominal load would have concluded the opposite. Figure 6.4 sets what each variant still completes beside what it spends completing it.

Safety is measured by injecting 14 defects. Seven come from inside the region Rust claims to govern. Seven were chosen because safe Rust is expected *not* to prevent them. Every one of the first seven is a compile error in Rust. It cannot be written at all. In C++ each is valid code. The defect appears only if a test drives it, and no single sanitiser covers them. The use-after-move is the case where no sanitiser reports anything: a moved-from object is valid but unspecified, so there is nothing to detect

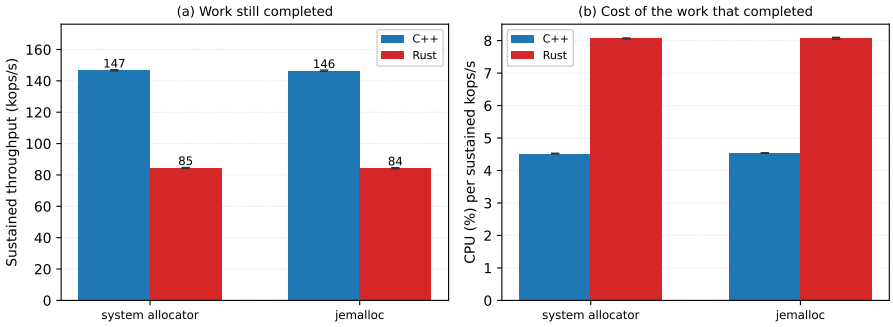


Figure 6.4: Behaviour at 784 kops/s offered, the deepest swept load at which every variant has stopped meeting the offered rate. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so that the two bars inside a group differ only in language. **(a)** What each variant still completes, where C++ sustains a factor of 1.7 more than Rust. **(b)** Total CPU divided by the work that actually completed, which is what rules out the reading that the variant delivering less is simply doing less. It spends 1.8 times as much CPU per sustained operation, and the CPU figures are summed across threads.

and the identity is simply lost. All seven boundary cases compile in safe Rust and misbehave. On two of them the C++ tooling is the one that reports. LeakSanitizer flags the reference cycle that Rust leaks in silence. UndefinedBehaviorSanitizer flags the overflow that Rust defines to wrap. ThreadSanitizer catches the lock-order inversion on some runs and not others, at a rate that is itself unstable. It builds its lock-order graph from acquisitions that succeed. A run whose first inversion blocks never closes the cycle. Two Rust model checkers do report both original boundary defects, Loom exploring interleavings soundly and Shuttle sampling the ones Loom cannot reach [46], but only after each was re-imported against the checker's own primitives and shrunk to model size. Miri reports the seam double release, the one class that safe Rust rejects and unsafe Rust does not, but it interprets Rust and cannot execute the C++ host the defect ships behind. That half of the boundary is caught at test time, and for the concurrency cases only after a rewrite. Figure 6.5 draws the whole catalogue as an outcome matrix.

Resource use separates the variants where latency could not. Two effects of comparable size sit in the resident set. The Rust build costs 1.4–2.5 MiB more peak resident set than the C++ build under either allocator. Linking `jemalloc` at all costs 1.5–2.4 MiB in either language. Both are real, and Figure 6.6 puts the pair beside the peak resident set size (RSS) they decompose. The language effect keeps its direction and order of magnitude on all three machines measured. The allocator's was an order of magnitude

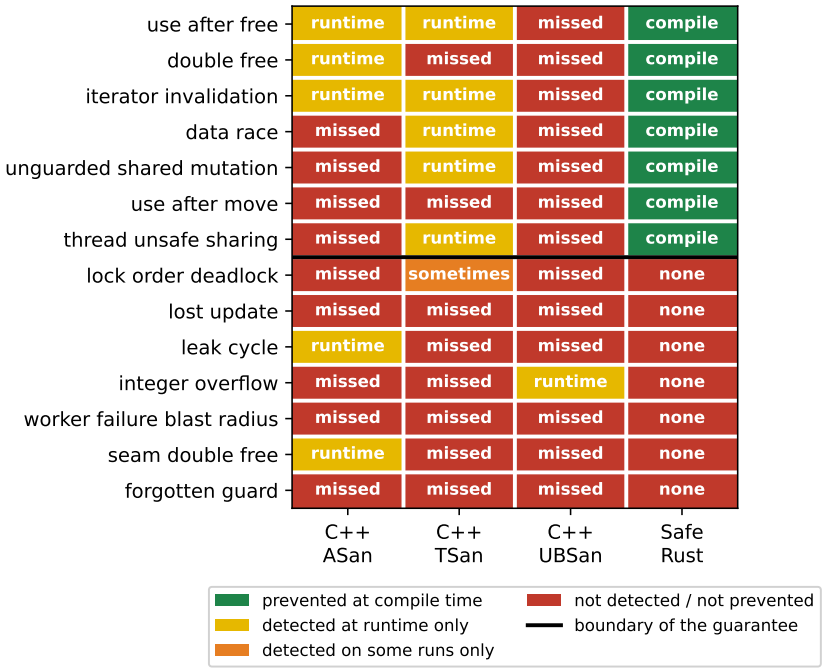


Figure 6.5: The fault-injection outcomes drawn as a matrix, generated from the taxonomy log the runner writes. Above the rule the Rust column is a single colour, because every defect there is a compile error and none can be written, while the C++ columns are a patchwork that no one sanitiser covers and the use-after-move row is covered by none of them. Below the rule the Rust column joins them, on the reference-cycle leak and the release-build overflow the C++ tooling is ahead, and the lock-order deadlock is the one cell where a tool reports intermittently, ThreadSanitizer flagging the inversion in 5 of 12 runs while the program deadlocks on every one.

larger on the virtualised host this study used first, so their ratio is a property of the machine. The language difference is a fixed cost, not a per-context one, and the quiet host locates it. `jemalloc`'s own accounting there puts +2.07 MiB more in arena bookkeeping and −0.16 MiB *less* in live allocation, and its resident bytes are −1.92 MiB lower while the process peak is +1.74 MiB higher. So the footprint sits outside the allocator, in the runtime a Rust process carries before it allocates. Counters at the operation scope explain the timing result instead: the Rust build retires 11% fewer instructions in 15% fewer cycles. CPU inverts with the regime. It is marginally lower for Rust while the service copes and much higher per completed operation once it does not. The migration seam is small on every axis measured: 11 C entry points in 231 lines of Rust. All raw-pointer work is confined to 9 functions. The core is left free of `unsafe`. No cost is detected against the same operation called natively. That bounds any per-operation tax at 10.91%, and Figure 6.7 shows the structure that bound belongs to. Three adoption costs beyond the seam were measured as well. Two of them contradict

the received objection. 54% of the Rust clean build is third-party code. The project's own source compiles in 0.8 times the CPU `g++` needs. The 6.8× binary-size gap is a linkage policy that reverses when the policy is held fixed. The compared component resolves 0 crates and contains 0 `unsafe` sites.

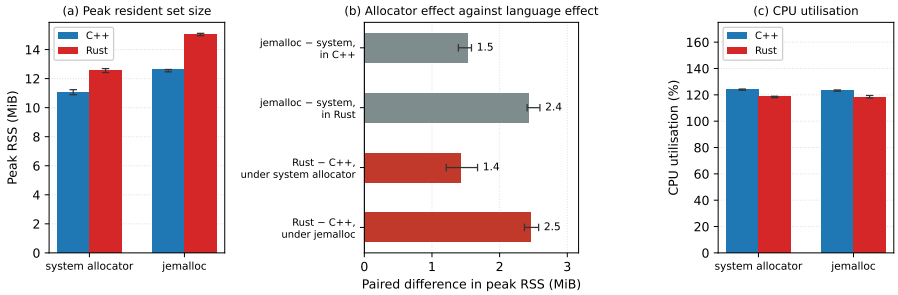


Figure 6.6: Resource use at a sustained offered load of 56 kops/s, grouped by allocator so that the two bars in a group differ only in language and the two groups differ only in allocator. Medians over 20 repeats with bootstrap 95% intervals. **(a)** Peak resident set size. **(b)** The two effects it contains, each as a paired difference over the same repeats, which on a shared axis are of comparable size here and were not on the host this study used first. **(c)** CPU utilisation, summed across threads, where the difference runs the other way and is small.

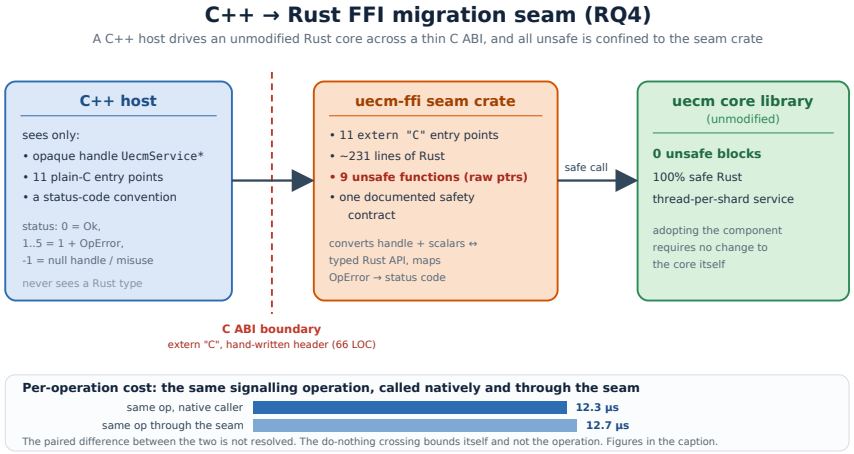


Figure 6.7: The C++-to-Rust FFI migration seam. The C++ host sees only an opaque handle, 11 plain-C entry points, and a status-code convention, while a thin `extern "C"` seam crate concentrates all of the raw-pointer `unsafe` in 9 functions and calls the unmodified Rust core, which remains free of `unsafe`. The band along the bottom is the comparison the argument rests on, the same signalling operation called natively at 12.3 µs and through the seam at 12.7 µs, whose paired difference is not resolved and is bounded at 10.91% of one operation. The do-nothing crossing costs 2.54 ns and bounds only itself.

7 Discussion

What a deployment should do with RQ1 depends on where it expects to sit, not on a single ranking. A service sized to run below every arm's knee should prefer the arm that holds its offered rate further. One that a signalling storm will drive past every knee should prefer the arm that still completes more work afterwards. Those two preferences point at opposite languages, which is why Chapter 6 reports both halves rather than a winner. The closest prior work measures this quantity for 5G control-plane procedures. It finds the same shape and traces it to stackless `async/await` [36]. That attribution is accepted here. The shape appeared with no asynchronous runtime on either side and the concurrency model held fixed. A runtime cannot be the only place such an effect originates. The prediction registered in advance also named where to look first, bounds and overflow checking. That part of it was wrong. Recording the miss is worth more to the next study than quietly substituting the explanation that survived.

On safety the difference is one of kind. The boundary is reported as a first-class result, not as a caveat. A compiler that refuses to build a program rules the defect out of every execution. A sanitiser rules it out only of the executions an instrumented test happens to take. What is absolute there is the execution coverage, not the breadth of the catalogue, and Figure 7.1 names the mechanism on each side of that line. Rust reasons about aliasing and not about acquisition order. A `Mutex` guard whose lifetime the borrow checker tracks perfectly says nothing about whether two threads take their locks in a compatible order [47]. The lost update escapes ThreadSanitizer for the same reason it escapes the borrow checker. Where every access is correctly synchronised, there is no unsynchronised access to find. Adopting Rust retires one category of concurrency defect and leaves untouched the category that fails silently. A migration plan reading “memory-safe” as “concurrency-safe” would mis-budget its review effort. One case reaches past the injected programs and narrows the credit due on both sides. A specification gap let an operation be submitted outside the service's running window. It survived every test, every parity vector and all three sanitisers because nothing ever called that path. That is the weakness of detection conditional on coverage. What turned undefined behaviour into a diagnosable failure on the Rust side was an ordinary bounds check, not a proof. The property worth budgeting for is “checked by default”, not “ruled out at compile time”. The root cause belongs to the method. A

prohibition stated without a mechanism behind it leaves each implementation to fail in its own idiom.

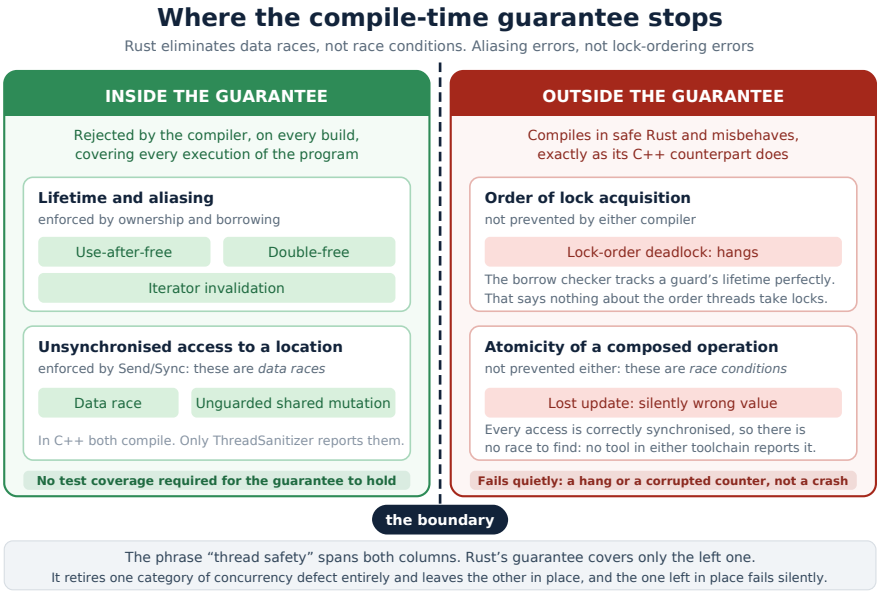


Figure 7.1: Where the compile-time guarantee stops. On the left are the classes safe Rust rejects outright, grouped by the *mechanism* that rejects them rather than by defect family, which is ownership and borrowing for lifetime and aliasing errors and the `Send`/`Sync` discipline for unsynchronised access to a location. On the right are the classes the catalogue included to find the edge, which neither compiler prevents. The diagram names one class per mechanism rather than the whole catalogue, the phrase "thread safety" spans both sides of the line, and the line itself moves once more than the diagram can show, because a program that writes `unsafe` crosses it voluntarily.

RQ3 splits by resource and the splits do not point the same way. RQ4 comes out cheap in the machine and expensive in the people. The memory penalty reproduces under both allocators and on a second machine with a different compiler. It is credible beyond its p-value. Peak resident set is a run-level maximum insensitive to the jitter that destroys a p99. It is a few MiB of fixed cost, and sized against the allocator's own effect the two are of the same order on both bare-metal hosts, interleaved on the reported one and with the allocator ahead on the other. On the virtualised host the allocator's was an order of magnitude larger. The language effect travels and the ratio does not. The quiet host also locates the cost. `jemalloc`'s own accounting puts it *outside* the allocator, because the Rust build holds less live data and fewer resident bytes there and still peaks +1.74 MiB higher. What a Rust process carries before it allocates is the standard library and the runtime. Which of its pages hold the difference is not attributed to a call site. One close study reports the opposite sign, with Rust lowest and the gap widening as concurrency rises [32]. The two are compatible. Their

C++ runs on Boost.Fibers, which allocates a stack per fiber. Their Rust runs on Tokio, where a task is a small heap-allocated future. What they measured is the per-context cost of two concurrency runtimes. Two further studies agree with the direction found here [34, 48]. On migration the dominant cost is human [49, 50]. Confining `unsafe` to the seam bounds what has to be reviewed, not what can be corrupted. Host and component share one address space [51, 52]. The build and footprint axes point the same way as the seam. An organisation worried about either should audit its dependency budget before blaming the compiler. The one axis where C++ leads outright is the edit-compile loop for changes that stay inside a translation unit.

The three hypotheses were committed before any measurement. Stating their disposition is what separates a test from a description. H1's median clause holds at the representative load. Its far-tail clause does not, and overload, which the wording never named, is where the languages actually part. The reported campaign now resolves that tail, in Rust's favour, and a second bare-metal host at another shard count agrees on the direction. H1 was right about where a below-saturation difference would show up and wrong that it was the only place. It was also not well posed. "Performance-equivalent" reads as one quantity and names at least three. H2 holds without exception for the seven injected classes of its kind. It fails in the looser reading its wording invites. It is accepted for memory safety and data-race freedom, rejected for concurrency correctness at large, and it is a claim about *safe* Rust only: the seam case shows the same class returning inside `unsafe`. H3 is supported by the bound on a cost the measurement could have missed, not by the failure to detect one. Three threats stay load-bearing behind all of it. One author wrote both implementations. The parity suite bounds behaviour and not performance. The end-to-end instrument measures the design faithfully and the language barely at all. Single-author bias is documented and not hypothetical: the closest matched pair in this industry reversed its own verdict across two benchmarking sessions, comparable after a code review and then Rust worse by 85–91% after further optimisation [33]. The overload divergence is replicated here with its mechanism narrowed and not closed, and no peer-reviewed study reports the same reversal.

The guidelines are the practical form of all this. Each carries the limit of the finding behind it. Choose a first component for the narrowness and the scalarity of its interface, not for its size. The redesign of a rich interface is the schedule risk. The boundary itself is not. Keep the C ABI in its own crate and the core free of `unsafe`. That keeps unsafety out of the dependency chains it otherwise propagates through [53], and makes

the reviewable surface a countable thing. Leave the concurrency review budget where it is. Prefer a shared-nothing structure in either language. The strongest concurrency result here belongs to the design and to neither compiler. Plan the learning curve as a training cost and instrument it. The second component can then be scheduled on measured effort. Measure the component as well as the round-trip. Sweep past the point where the service stops coping. That is the guideline this study would most like to have followed from the start.

8 Conclusion and Future Work

The thesis compared modern C++ and Rust on a UE Context Management service modelled on a real gNB-CU-CP component. It looked at four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Both implementations were built against one frozen specification and driven along identical code paths. That is what lets a language effect be told apart from a difference of design or of skill.

On safety the difference is one of kind and it has a clear boundary. The edge stops where the catalogue says it does. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time. Hazards that C++ can only hope to catch with instrumented tests become programs that cannot be written. All seven boundary classes compile and misbehave in safe Rust as they do in C++. On two of them the C++ tooling reports what Rust's does not. Stated precisely, Rust eliminates data races and aliasing errors. It does not eliminate race conditions, lock-ordering errors, leaks, wrapping arithmetic or the blast radius of a failed worker, and it eliminates nothing inside an `unsafe` block.

On performance the evidence first divides by what is being measured. An end-to-end operation here is 0.250% language. The rest is an operating-system handoff. An end-to-end null bounds nothing about either compiler. The same operation measured apart is 1.6× faster in Rust. At the representative load the paired test resolves 1.3 μs in Rust's favour, too small for a deployment to care about at the median. The far tail on the reported host orders the languages in the same direction. A virtualised host could not settle that percentile. The Rust arms then hold their offered rate to 448 kops/s against 392, and once all four have collapsed C++ sustains 1.7× the throughput of Rust, which spends 1.8× the CPU on each operation it completes. On memory the Rust build carries a few MiB more fixed footprint under both allocators, in the same direction and order of magnitude on all three machines measured. The allocator choice costs about as much on both bare-metal hosts, and an order of magnitude more on the virtualised one this study used first.

What follows is a narrow claim, narrower than this thesis expected to make. An unmodified Rust core can be driven from C++ across a thin seam. The cost on a

real operation is bounded at 10.91%. All `unsafe` is confined to the seam. The core is verified free of it. For stateful, concurrent, partitionable control-plane services of this kind, at the loads where the service still meets its offered rate, the choice is which classes of defect an organisation wants to make unwriteable, and whether it will pay the engineering and learning cost of getting them. Past that point it is a performance decision as well. What the thesis adds is an instrument, not a verdict: a frozen specification, a parity-tested pair of implementations, a measurement framework that reaches past the saturation knee, a decomposition that says how much of what it measures the language is answerable for, and a defect catalogue that maps where the guarantee stops.

Future work answers the limitations one at a time. The immediate priority needs no new machine: vary the transparent-hugepage policy on one host with everything else held fixed. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that compares against the cost of the allocator, which the virtualised host put an order of magnitude higher and neither bare-metal host does. The hugepage policy is the candidate this study can name but not test, because it differed between the hosts and only the system-allocator arm inherits it. Beyond that, replaying anonymised production traces would validate the operation mix. A subsystem with genuinely shared mutable state would exercise the two languages differently than a shared-nothing design does. Evaluating Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [36]. Measuring what in-process isolation of the two heaps costs would price what protecting an `unsafe`-free core is worth [51]. A longer case study tracking defect density and review effort during an incremental adoption is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, through the Cursor development environment, is the only generative AI system used in preparing this thesis, from June 2026 until submission. It is named neither as an author, which University of Helsinki policy forbids, nor as a source, which is the author's own choice.

What the assistance covered. It was extensive. The model drafted and revised the running prose and proposed the order in which the arguments are made. It wrote and refactored both implementations of the UE Context Management service, the load generator, the fault-injection catalogue, the interoperability prototype, the analysis code behind the bootstrap intervals and the paired permutation tests, and the figure-generating, build and LaTeX sources. It also suggested candidate literature. The abstract is the one part it must not touch, because it is this programme's maturity test, and University of Helsinki policy never permits a language model in one. The abstract was accordingly written by the author unaided.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the control on which the study's value rests are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only systematic difference between the two implementations. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names, and the quantities the thesis quotes reach the page as macros regenerated from the raw logs. The readings are the author's too, including the restraints the evidence imposes: a sweep dominated by cross-thread handoff cannot bound an effect of the language, an unresolved comparison is not a demonstration of equivalence, and an operating point noticed after the data was in hand is described rather than tested.

How the output was checked. Model output was treated as a draft to be falsified, and all of it, prose and code alike, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. Most were then graded by how far the claim made of it could be traced: to the page or clause itself, to a publisher-deposited record carrying it, or no further than confirmed

metadata behind a paywall. The audit records that grade per entry, so an entry whose supporting passage was never opened is listed rather than absorbed. It found four passages that overstated or misplaced their source, one a quotation absent from the work it was attributed to, and all four were corrected. The audit is kept beside the artefacts. Code was checked by running it against a conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in one predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again in checking, and the defensible parts of this thesis are the parts checked against a source, a specification, or a measurement.

Responsibility. The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[14] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes

that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L,

2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. "Firecracker: Lightweight Virtualization for Serverless Applications". In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434. URL: https://www.usenix.org/conference/nsdi20/presentation/agache.

[23] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[24] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[25] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[26] Cybersecurity and Infrastructure Security Agency. *Product Security Bad Practices*. Joint Guidance. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt. CISA and FBI, Jan. 2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices.

[27] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language De-*

sign and Implementation (PLDI). 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[28] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[29] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[30] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[31] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[32] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[35] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[36] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[37] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[38] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[42] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[43] S. Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[44] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[45] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. “OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck”. In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[46] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. “Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[47] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. “Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs”. In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[48] A. Sundvisson and S. Hofer. “User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust”. Bachelor’s thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[49] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. “Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study”. In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[50] W. Crichton and S. Krishnamurthi. “Profiling Programming Language Learning”. In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[51] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[52] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

[53] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.